

LAB 10

JAVASCRIPT 3: APIs

What You Will Learn

- Using additional language features of JavaScript
- Fetching data from web APIs using JavaScript
- Working with Promises and `async` and `await`
- Using browser APIs and external APIs.

Approximate Time

The exercises in this lab should take approximately 120 minutes to complete.
This is a long lab!

Fundamentals of Web Development, 3rd Ed

Randy Connolly and Ricardo Hoar

Textbook by Pearson
<http://www.funwebdev.com>

Date Last Revised: August 22, 2025
Revised: Exercise 10.2 (added private members), 10.3 (importing a JSON file)
Revised: Added Exercise 10.15 (charting)
Replaced: Test Your Knowledge #5

PREPARING DIRECTORIES

The starting `lab10` folder has been provided for you (within the zip folder downloaded from Gumroad).

*Note: these labs use the convention of blue background text to indicate filenames or folder names and **bold red** for content to be typed in by the student.*

ARRAY FUNCTIONS

JavaScript provides many powerful iterative functions that take functions as parameters. While a bit confusing at first, you will be using them frequently in future labs and chapters.

Exercise 10.1 — USEFUL ARRAY FUNCTIONS

- 1 Examine `lab10-ex01.html` and then edit `js/lab10-ex01.js` by adding the following code.

```
const sortedTitles = titles.sort( );
console.log(sortedTitles);
```

Since titles is simply an array of strings, it can be easily sorted. Note that this function mutates the original array. You can use `.toSorted` if you do not want to change the original array.

- 2 To sort an array of objects is more complicated. We have to create a compare function; to do so, add the following code (and then test).

```
const paintingsSortedByYear = paintings.sort( function(a,b) {
  if (a.year < b.year)
    return -1;
  else if (a.year > b.year)
    return 1;
  else
    return 0;
});
console.log(paintingsSortedByYear);
```

We can make this even more concise.

- 3 Comment out the code from the previous step and enter the following, more concise version of the same using the ternary operator and arrow functions:

```
const paintingsSortedByYear = paintings.sort( (a,b) => {
  return a.year < b.year ? -1 : 1;
});
```

- 4 Test the following version which is even more concise.

```
const paintingsSortedByYear =
  paintings.sort( (a,b) => a.year < b.year );
```

- 5 To create a new array that is a subset of an existing array, you can use the `filter()` method. Add the following and test.

```
const nineteenthCentury = paintings.filter(
  painting => painting.year >= 1800 && painting.year < 1900);
console.log(nineteenthCentury);
```

Notice that you supplied the conditional test that must be true for a painting to be included in the new array.

- 6 To find the first element in an array that matches a criteria, you can use the `find()` method. Add the following and test.

```
const manet =
  paintings.find( painting => painting.artist == "Manet");
console.log(manet);
```

Notice it finds only the first painting by Manet.

- 7 Finally, this function approach can also be used for iteration via the `forEach()` method of arrays. Add the following and test.

```
paintings.forEach(
  p => { console.log(p.title + " by " + p.artist) });
```

- 8 Add the following code and test. Examine the error message in the console.

```
let indexBronz = 0;
paintings.forEach( p => {
  if (p.artist == "Bronzino") break;
  indexBronz++;
});
console.log("index of Bronzino="+indexBronz);
```

*Unlike a regular for loop, you **cannot** break from a `forEach()`.*

- 9 Modify the previous step's code as follows and test.

```
let indexBronz = 0;
paintings.forEach( (p,index) => {
  if (p.artist == "Bronzino") indexBronz = index;
});
console.log("index of Bronzino="+indexBronz);
```

This version uses the optional index parameter for `forEach`. After you sort the paintings, the painting by Bronzino is the second in the array.

- 7 You can perform more powerful filtering and finding by using regular expressions. Add the following and test.

```
// first define regular expression pattern ('i' = case insensitive)
const re = new RegExp('ea','i');
const results = paintings.filter( p => p.title.match(re) );
console.log(results);
```

This uses regular expressions to search for all paintings whose title contains the word 'ea'.

- 8 Finally, a powerful new addition to JavaScript is the spread operator, which can be used to expand an array or object. This sounds pretty confusing, so let's look at some examples. Try entering the following and test.

```
// using the spread operator
```

```
const moreTitles = ["Balcony", "Sunflowers", ...titles];
console.log(moreTitles);
```

This has added the contents of the titles array (defined at the top of the file) to the end of the moreTitles array.

- 9 To make sense of this, let's implement that same functionality in the old-fashioned way:

```
const moreTitlesOldFashioned = ["Balcony", "Sunflowers"];
for (let t of titles) {
  moreTitlesOldFashioned.push(t);
}
console.log(moreTitlesOldFashioned);
```

As you can see, using the spread operator simplifies our code (well, that is for you to decide; it certainly uses less code).

- 10 A common task with arrays is to create a new array based on data within another array. For instance, enter the following and test.

```
console.log('- create new array from existing array of objects');
const artists1 = [];
for (let painting of paintings) {
  artists1.push(painting.artist);
}
console.log(artists1);
```

This creates an array of artist names based on artist field in your array of paintings.

- 11 A more concise way of doing this is via the `map()` function, show below. Try adding this code and test.

```
const artists2 = paintings.map( function (painting) {
  return painting.artist;
});
console.log(artists2);
```

- 12 An even more concise way of doing this is via arrow syntax, show below. Try adding this code and test.

```
const artists3 = paintings.map( painting => painting.artist );
console.log(artists3);
```

- 13 The main reason for using `map` is to transform one array into another of the same size. Try this by adding this code and test.

```
console.log('- using map to transform an array');
const artistList = paintings.map( p => `<li>${p.title}</li>` );
console.log(artistList);
```

The new array is an array of strings.

- 14 Now let's instead create an array of DOM nodes that we will populate the existing `<ul>` element:

```
const artistDOMList = paintings.map( p => {
  const li = document.createElement('li');
  li.textContent = p.title;
  li.value = p.id;
  return li;
});

const list = document.querySelector("#list");
artistDOMList.forEach( li => { list.appendChild(li)});
```

- 15 Finally, another useful array function is the `reduce` function, which transforms an array into a scalar value. Enter the following code and test.

```
let total = sales.reduce((subtotal,current) => subtotal + current)
console.log("total=" + total);
```

TEST YOUR KNOWLEDGE #1

Using the files `lab10-test01.html` and `lab10-test01.js`, solve the following code problems two ways: first using both regular loops and second using the appropriate array function. Assume your data is the `stocks` array shown below :

```
const stocks = [
  {symbol: "AMZN", name: "Amazon", price: 23.67, units: 59},
  {symbol: "AMT", name: "American Tower", price: 11.22, units: 10},
  {symbol: "CAT", name: "Caterpillar Inc", price: 9.00, units: 100},
  {symbol: "APPL", name: "Amazon", price: 234.00, units: 59},
  {symbol: "AWK", name: "American Water", price: 100.00, units: 10}
];
```

- 1 Add a new function property named `total` to each stock object which is equal to `price * units` via a `for...of` loop and then using the `forEach()` array function. Adding this function will require a function expression.
- 2 Find the first element whose symbol name is "CAT" (using loop and then using `find()` function).
- 3 Create a new array that contains stocks whose price is between \$10 and \$25 (using a `for` loop and then using `filter()` function).
- 4 Create a new array of strings containing `<li>` elements displaying the stock `name` property (using a `for` loop and then using `map()` function).

PROTOTYPES, CLASSES, AND MODULES

Prototypes are a more memory efficient mechanism for adding functions to objects. Instead of adding functions to each object instance, you can add the function once to its `prototype` property.

Exercise 10.2 — USING PROTOTYPES AND CLASSES

- 1 Edit `js/lab10-ex02.js` by adding the following:

```
// simple luminance (brightness) calculation
Color.prototype.luminance = function () {
  let r = this.rgb[0];
  let g = this.rgb[1];
  let b = this.rgb[2];
  return 0.2126*r + 0.7152*g + 0.0722*b;
}
```

Source: https://en.wikipedia.org/wiki/Relative_luminance

- 2 Test this function by adding the following code:

```
const light = new Color("light", "#f2f1e6", [250,247,230] );
console.log( light.luminance() );
const mid = new Color("mid", "#aba798", [171,167,152] );
console.log( mid.luminance() );
const dark = new Color("dark", "#231f20", [45,31,32] );
console.log( dark.luminance() );
```

- 3 Test by viewing `lab10-ex02.html` in browser.

The provided HTML demonstrates these colors within boxes, but you will have to examine the console to see the luminance values.

- 4 Use this function by adding the following at the bottom of your code:

```
// add loop here
colors.forEach( (c) => {
  console.log(c.name + ', ' + c.luminance());
});
```

- 5 Test. You will need to examine the console.

- 6 Modify the loop as follows:

```
colors.forEach( (c) => {
  createBox(c.hex, c.luminance());
});
```

- 7 Add the following function and test:

```
function createBox(hex,luminance) {
  const div = document.createElement('div');
  div.classList.add('box');
  div.style.backgroundColor = hex;
  div.textContent = luminance.toFixed(2);
  document.querySelector("#list").appendChild(div);
}
```

- 8 Notice that the black text is difficult to read against a dark background. Modify your new function as follows and then test.

```
function createBox(hex, luminance) {
  const div = document.createElement('div');
  div.classList.add('box');
  div.style.backgroundColor = hex;
  if (luminance < 80) div.style.color = 'white';
  div.textContent = luminance.toFixed(2);
  document.querySelector("#list").appendChild(div);
}
```

- 9 Comment out the `Color` constructor function, the `luminance` prototype function, and the prototype testing code. Add the following.

```
// add your class definition here ...
class Color {
  constructor(name, hex, rgb) {
    this.name = name;
    this.hex = hex;
    this.rgb = rgb;
  }

  luminance() {
    let r = this.rgb[0];
    let g = this.rgb[1];
    let b = this.rgb[2];
    return 0.2126*r + 0.7152*g + 0.0722*b;
  }
}
```

- 10 Test. It should work exactly the same.

Classes are just syntactical sugar, meaning that your `Color` class is simply an alternate way of creating a function constructor with prototype functions.

- 11 Modify the class constructor as follows.

```
class Color {
  // # before the name indicates it is private member
  #name = '';
  #hex = '';
  #rgb = '';
  constructor(name, hex, rgb) {
    this.#name = name;
    this.#hex = hex;
    this.#rgb = rgb;
  }

  luminance() {
    let r = this.#rgb[0];
    let g = this.#rgb[1];
    let b = this.#rgb[2];
    return 0.2126*r + 0.7152*g + 0.0722*b;
  }
}
```

12 Test.

Notice that the reference to `c.hex` in `createBox` (that is, outside the class) now returns `undefined` since it is now private to the class. If we want access to the value of a private member, you will have to define getters.

13 Modify the class constructor as follows (some code omitted) and test.

```
class Color {
  // # before the name indicates it is private member
  #name = '';
  #hex = '';
  #rgb = '';
  constructor(name, hex, rgb) {
    ...
  }

  get hex() { return this.#hex }
  get name() { return this.#name }
```

The getter provides access to a member as a property. The reference to `c.hex` in `createBox` will now work.

JavaScript class syntax also supports static members and inheritance. If you are interested, you are encouraged to examine the MDN JavaScript documentation (<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes>).

Exercise 10.3 — USING MODULES

- 1 Edit `js/color.js` and examine. It contains code from the previous exercise. Add the following.

```
const getSampleColors = () => sampleColors;

export default getSampleColors;
```

- 2 Edit `js/lab10-ex03.js` by adding the following:

```
// import required function from your module
import getSampleColors from './color.js';

document.addEventListener("DOMContentLoaded", () => {
  // use that function
  getSampleColors().forEach( (c) => {
    createBox(c.hex, c.luminance());
  });
  ...
})
```

- 3 Edit `lab10-ex03.html` by adding the following. Test.

```
<head>
  <meta charset="utf-8"/>
  <title>Lab 10</title>
  <script src="js/color.js" type="module"></script>
  <script src="js/lab10-ex03.js" type="module"></script>
  <link rel="stylesheet" href="css/lab10-ex03.css">
</head>
```

Notice that modules can only be used within a module. Thus both the module `color.js` and the file using the module (`lab10-ex03.js`) must be marked with `type=module`.

- 4 Instead of having the `createBox` function outside the class, we could put it within it. Modify the class in `color.js` by moving the `createBox` function from `lab10-ex03.js` and editing it follows:

```
class Color {
  ...
  createBox(container) {
    const div = document.createElement('div');
    div.classList.add('box');
    div.style.backgroundColor = this.hex;
    if (this.luminance() < 80) div.style.color = 'white';
    div.textContent = this.luminance().toFixed(2);
    container.appendChild(div);
  }
}
```

Instead of hard-coding the container, we are now passing it in. Notice also the use of the `this` keyword here.

- 5 Modify the loop in `lab10-ex03.js` as follows and test.

```
document.addEventListener("DOMContentLoaded", () => {
  // use that function
  const container = document.querySelector("#list");
  getSampleColors().forEach( (c) => {
    c.createBox(container);
  });
});
```

- 6 There are a variety of different ways of making content available from a module. Modify `color.js` as follows.

```
//export default getSampleColors;
export { Color, getSampleColors };
```

Instead of using default keyword, which only allows a single item to be exported, here you can supply several objects.

- 7 Modify `lab10-ex03.js` as follows and test. Everything should work as before.

```
//import getSampleColors from "./color.js";
import { getSampleColors } from "./color.js";
```

- 8 Modify `lab10-ex03.js` as follows. Notice the use of a wildcard. This will let us use everything that was exported without requiring an explicit import for each item.

```
//import getSampleColors from "./color.js";
//import { getSampleColors } from "./color.js";
import * as colors from "./color.js";
```

- 9 Modify the code using `getSampleColors` as follows then test. It should work.

```
colors.getSampleColors().forEach( (c) => { ...
```

- 10 Add the following code to the top of `lab10-ex03.js` and test.

```
import * as colors from "./color.js";
import galleries from './galleries.json' with { type: 'json' };
console.dir(galleries);
```

This provides a simpler method for reading in and parsing a JSON file.

TEST YOUR KNOWLEDGE #2

You will be modifying a file named `gallery.js`. This is going to be a module that will be used in your other files. It already has some sample data in it.

- 1 In this module, create a JavaScript class named `GalleryItem` that represents a gallery list item. Its constructor should take two arguments: the gallery id and the gallery name.
- 2 Add a method/function to the class named `render()` that returns a DOM element that represents a `<li>` element. The `textContent` of the element should be the gallery name. Add an attribute named `data-id` which is set to the gallery id.
- 3 Add a function to the module named `getSampleGalleries()` which returns the `galleries` array. Export both `GalleryItem` and `getSampleGalleries` at the end of the module.
- 5 Modify the file `lab10-test02.js` so that it imports `getSampleGalleries` from the gallery module. Use the `getSampleGalleries()` function to retrieve the sample data.
- 6 Loop through the sample data. For each `GalleryItem`, add the element returned from its `render()` method to the `<ul>` list using `appendChild()`.
- 7 Add in the necessary `<script>` tags to `lab10-test02.html`. The result should look similar to that shown in Figure 10.1.

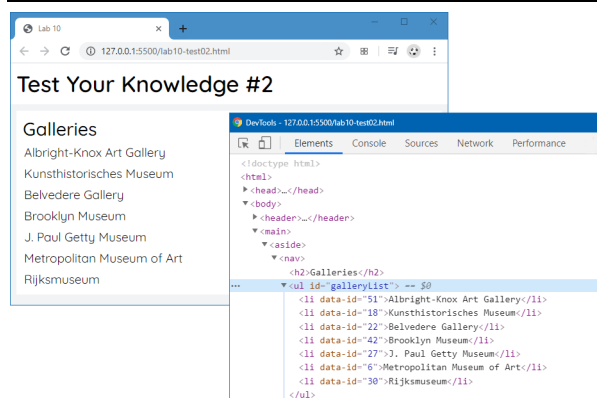


Figure 10.1 – Finished Test Your Knowledge #2

ASYNCHRONOUS CALLS TO WEB APIs

In this section you will use JavaScript to access web APIs (also referred to as web services). This used to require using JQuery because the native approach was quite complicated. In recent years however, browsers now support the native `fetch` method which we will use here.

Exercise 10.4 — USING FETCH

- 1 Examine `https://api.github.com/orgs/funwebdev-2nd-ed/repos` in the browser.

Web APIs are simply web pages that return JSON instead of HTML.

- 2 Examine `lab10-ex04.html` and then edit `js/lab10-ex04.js` by adding the following code within the supplied `DOMContentLoaded` event handler. Test and examine the console.

```
let url = 'https://api.github.com/orgs/funwebdev-2nd-ed/repos';
// use fetch to request data from an API
let foo = fetch(url);
console.log(foo);
```

Fetch returns a Promise, which is a way to handle asynchronous operations. To access the data from the fetch, we have to wait until it is received, which we can do via `then()`.

- 3 Modify the code as follows and test.

```
let foo = fetch(url);
let bar = foo.then( response => { console.log(response) } );
console.log(bar);
```

When fetch receives a response, it will call the callback function passed to `then()`. Notice that the actual return data is hidden in the response body and will require some additional work. Notice also that the `then()` function also returns a promise.

*As a result, the `console.log(bar)` is executed **before** the `console.log(response)`.*

- 4 Modify the code as follows and test.

```
let url = 'https://api.github.com/orgs/funwebdev-2nd-ed/repos';
fetch(url)
  .then( response => response.json() )
  .then( data => { console.log(data); } );
console.log('after the fetch');
```

It is common to chain calls to `then()` together, thereby eliminating the need for additional variables.

The received data is now being received. Notice also that the data display happens after the other code has executed.

5 Modify the URL in the fetch so that it is incorrect and then test.

Not surprisingly, this generates a run-time error. Our fetch needs some additional work to handle errors.

6 Add the following code and test.

```
fetch(url)
  .then( response => response.json() )
  .then( data => { console.log(data); } )
  .catch( err => { console.log('Handle error somehow') });
```

Here we simply have an error message; in a real example would likely need to notify the user. You can supply more information

7 Modify the first then() as follows and test.

```
fetch(url)
  .then( response => {
    if (response.ok) {
      return response.json();
    } else {
      return Promise.reject({
        status: response.status,
        statusText: response.statusText
      })
    }
  })
  .then( data => { console.log(data); } )
  .catch( err => { console.log('err='+err) });
```

While not necessary, there are occasions when you need to customize your error handling in this way.

8 Restore the fetch URL to the correct one and test.**9** Comment out the code in step 7 and replace with the following.

```
const url3 = './js/galleries.json';
fetch (url3)
  .then (resp => resp.json())
  .then (data => {
    console.log(data);
  })
  .catch ( err => { console.error(err) });
```

You can also use fetch to load a local JSON file. This can be helpful when an external API is not ready yet or is unavailable. Once it is ready or available, you simply need to replace the file reference with the correct URL.

In the next exercise, you will modify the DOM based on fetched data.

Exercise 10.5 — FETCHING THEN MODIFYING THE DOM

- 1 Examine the url specified within `lab10-ex05.js` in the browser.
- 2 Add the following code in `lab10-ex05.js` as follows and then test.

```
// fetch the continents from the api in url
fetch(url)
  .then( (resp) => resp.json() )
  .then( data => { displayContinents(data) } )
  .catch(error => console.error(error));
```

To simplify the code for explanatory purposes, we are simplifying the error handling.

- 3 Add the following function to the code.

```
function displayContinents(continents) {
  const list = document.querySelector('.box ul');

  for (let c of continents) {
    const item = document.createElement('li');
    item.textContent = c.name;
    list.appendChild(item);
  }
}
```

- 4 Test in browser. The result should look similar to that in Figure 10.2.

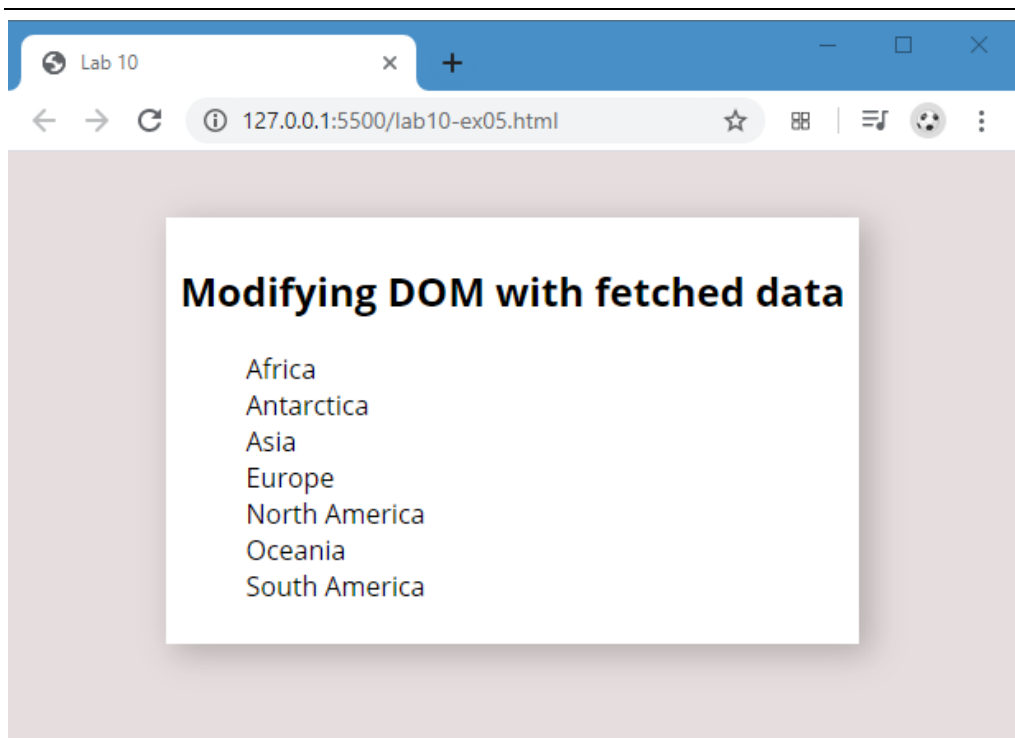


Figure 10.2 – Finished Exercise 10.5

- 5 You are now going to try modifying the code in various ways that won't work. The purpose is to illustrate some common mistakes that students make using fetch.

Change your fetch code as follows and test.

```
// this doesn't work ... why not?
let fetchedData;
fetch(url)
  .then( (resp) => resp.json() )
  .then( data => {
    // save fetched data in global variable
    fetchedData = data;
  })
  .catch(error => console.error(error));
// this looks like it should work but it doesn't
displayContinents(fetchedData);
```

Examine the error message in the console. Can you figure out why it didn't work?

- 6 To help understand the nature of the error, add the following console messages and test.

```
console.log('before fetch');
let fetchedData;
fetch(url)
  .then( (resp) => resp.json() )
  .then( data => {
    // save fetched data in global variable
    fetchedData = data;
    console.log('got the data');
  })
  .catch(error => console.error(error));
// this looks like it should work but it doesn't
console.log('after fetch');
console.log(fetchedData);
displayContinents(fetchedData);
```

Notice that the message 'after fetch' appears before message 'got the data'. In other words, fetchedData is undefined still. Remember the fetch takes time and is asynchronous.

- 7 Remove the changes made in steps 5 and 6, and add the following code after the fetch. Here we are trying to set up click event handlers for the new elements added within displayContinents().

```
// this doesn't work either... why not?
const items = document.querySelectorAll('.box ul li');
console.log(items);
for (let li of items) {
  li.addEventListener('click', (e) => {
    console.log(e.target.textContent);
  })
}
```

Examine the output in the console. Can you figure out why it didn't work? It's the same problem as the previous one. We are trying to set event handlers for items that don't exist yet (they won't exist until the data is received).

- 8 To make this work, you will need to move this code to the fetch as shown below.

```
fetch(url)
  .then( (resp) => resp.json() )
  .then( data => {
    // create the list items based on received data
    displayContinents(data);

    // set up event handlers
    const items = document.querySelectorAll('.box ul li');
    for (let li of items) {
      li.addEventListener('click', (e) => {
        console.log(e.target.textContent);
      })
    }
  })
  .catch(error => console.error(error));
```

- 9 Test. It should work correctly now.

One of the early common uses of asynchronous API calls was to create autocomplete lists. As the user types in content in a text box, a dynamic list is displayed showing matches based on the letters being typed by the user. The next exercise illustrates this functionality.

EXERCISE 10.6 — CREATING AN AUTOCOMPLETE BOX

- 1 Examine `lab10-ex06.html` and then edit `js/lab10-ex06.js` by adding the following code after the endpoint URL definition:

```
const endpoint = ...

// begin with an empty universities array
const universities = [];

// fetch from API will populate this array using spread operator
fetch(endpoint)
  .then(response => response.json())
  .then(data => universities.push(...data))
  .catch(error => console.error(error));
```

- 2 The next step is to define keyboard event handler for the `<input>` element after the fetch. This handler can be defined outside the fetch since the user interaction will occur considerably later in time.

```
// now set up keyboard event handlers
const searchBox = document.querySelector('.search');
const suggestions = document.querySelector('#filterList');
searchBox.addEventListener('input', displayMatches);
```

3 Create the `displayMatches()` function as follows:

```
// handler for keyboard input
function displayMatches() {
  // don't start matching until user has typed in two letters
  if (this.value.length >= 2) {
    const matches = findMatches(this.value, universities);

    // first remove all existing options from list
    suggestions.replaceChildren();

    // now add current suggestions to <datalist>
    matches.forEach(univ => {
      let option = document.createElement('option');
      option.textContent = univ.name + ', ' + univ.city;
      suggestions.appendChild(option);
    });
  }
}
```

4 Finally create the `findMatches()` function as follows:

```
// uses filter and regular expression to create list of matching
universities
function findMatches(wordToMatch, universities) {
  return universities.filter(obj => {
    const regex = new RegExp(wordToMatch, 'gi');
    return obj.name.match(regex);
  });
}
```

5 Test in browser. Try typing in the first three letters of 'Alaska'. Notice how it matches any university that contains the letter 'ala' in the name. As you type more, the list should shrink.

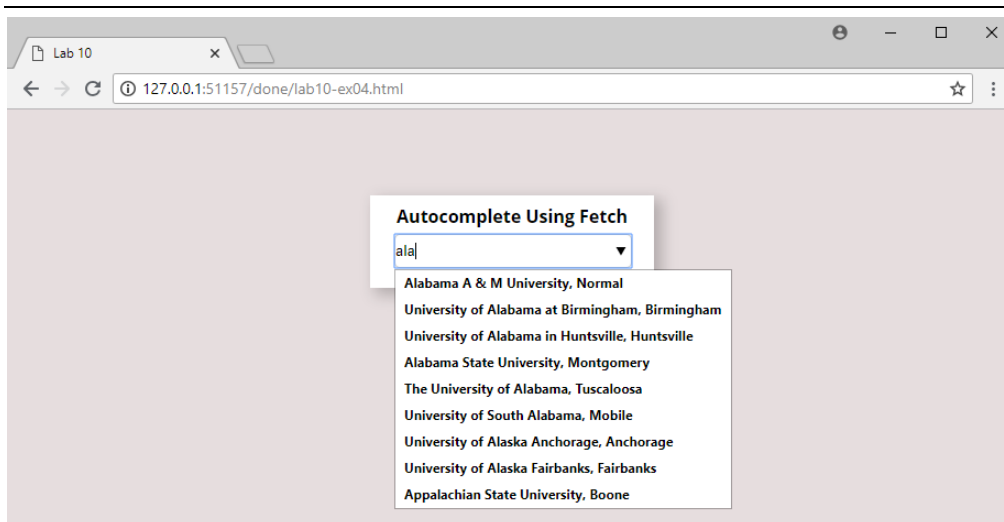


Figure 10.3 – Completed AutoComplete Box

Sometimes your fetch will need to send information to the server. In the next exercise, you will use fetch to POST data.

EXERCISE 10.7 — POSTING USING FETCH

- 1 Examine `lab10-ex07.html` in the browser then examine `js/paintings.js` which is a module that contains painting data and has a method for outputting the markup for a single painting. In this exercise, you will be adding functionality to the Add to Favorites buttons.
- 2 You will editing `js/lab10-ex07.js` in the rest of this exercise and will be adding the your code within the `DOMContentLoaded` handler. Begin by adding the following function:

```
function showSnackBar(message) {
  const snack = document.querySelector("#snackbar");
  snack.textContent = message;
  snack.classList.add("show");
  setTimeout( () => {
    snack.classList.remove("show");
  }, 3000);
}
```

A snackbar is a brief message that appears at the top or bottom of a screen.

- 3 Instead of adding individual event handlers for each button, you will use event delegation. Add the following code:

```
// set up button handlers here using event delegation
document.querySelector('main').addEventListener('click', (e) => {
  if (e.target.nodeName.toLowerCase() == 'button') {
    // retrieve data from button and
    let id = e.target.getAttribute('data-id');
    // get painting object for this button
    let p = paintings.find( p => p.id == id);
  }
})
```

If you inspect the generated HTML you will see that each button has a `data-id` attribute which contains its `id` property.

- 4 Your button is going to `POST` the painting `id` and `title` to the external API. One way to construct the data the page will be posting is to use the JavaScript `FormData` object. Add the following within the `if` statement added in the previous step.

```
// We will post the painting id and title to favorites
let formBody = new FormData();
formBody.set("id",p.id);
formBody.set("title",p.title);
```

- 5 In order to have `fetch` make a `POST` request, you will need to construct an options parameter that contains the data to post. Add the following after code from the previous step.

```
const opt = {
  method: 'POST',
  body: formBody
};
```

- 6 Add the following after code from the previous step.

```
// now let's post via fetch
fetch(url, opt)
  .then( resp => resp.json() )
  .then( data => {
    // display notification it was successful
    showSnackBar(`${data.received.title} was added to
favorites`);
  })
  .catch( error => {
    showSnackBar('An error occurred, favorites not
modified');
  });
}
```

The Web API you are fetching returns a simple JSON object with two properties: *message* and *received* (see Figure 10.5 below).

- 7 Test. The result should look similar to that in Figure 10.4.

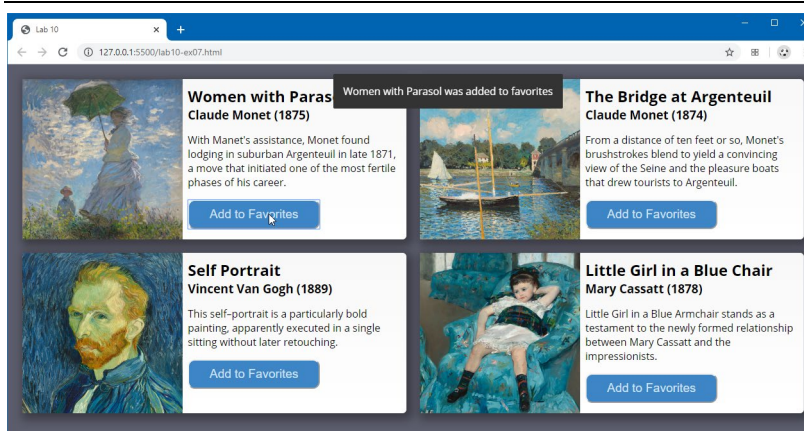


Figure 10.4 – Completed exercise 10.7

While you can easily examine a web API that responds to GET requests within your browser, it is more difficult to examine one that expects POST requests (or PUT and DELETE requests). Many developers make use of a third-party tool such as Postman or Insomnia, or a website such as <https://hoppscotch.io/>. Figure 10.5 illustrates how to use the free hoppscotch.io site to make a POST request to the API used in the last exercise. Notice that the Content Type needed to be set along with the parameters. Notice also the JSON returned from the API.

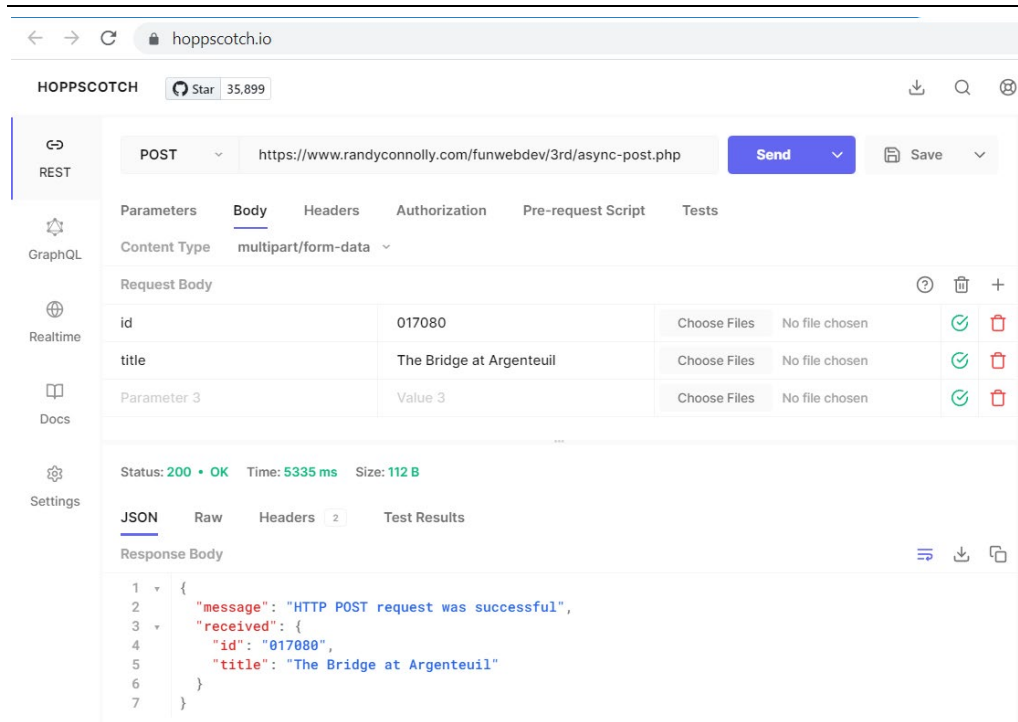


Figure 10.5 – Examining a POST web API

Fetching takes time. A common user interface feature is to supply the user with a loading animation while the data is being fetched. The next exercise will illustrate how this works.

EXERCISE 10.8 — ADDING A LOADING ANIMATION

- 1 Examine `lab10-ex08.html` in the browser. Notice the empty `<div>` with the id of `movieLoading`, which contains the CSS loading animation. This example also fetches a very large data set and might take 20-30 seconds to load.
- 2 Add the following code just before the fetch to `js/lab10-ex08.js`.

```

// hide form and display loading animation
document.querySelector("form.box").style.display = "none";
document.querySelector("#movieLoading").style.display =
  "block";

```

- 3 Add the following code within the fetch.

```
fetch(endpoint)
  .then(response => response.json())
  .then(data => {
    // show form and hide animation
    document.querySelector("form.box").style.display =
      "block";
    document.querySelector("#movieLoading").style.display =
      "none";
    movies.push(...data);

  } )
  .catch(error => console.error(error));
```

- 4 Test. The loading animation should work.

TEST YOUR KNOWLEDGE #3

In this exercise, you will be working with nested fetches. You will be modifying `lab10-test03.js` and consuming two APIs. The URLs for the two APIs and the location of the photo image files are included in the JavaScript file. The result will look similar to that shown in Figure 10.6.

- 1 Create the first fetch after the user clicks the first button; use the fetched country information to populate the `<select>` list. Be sure to first empty the `<select>` element (by calling its `replaceChildren()` method; otherwise each time the button is clicked, the list size will grow. For each `<option>` element, set its `value` attribute to the `iso` property of each country object.
- 2 Display the first loading animation and hide the `<main>` element while the fetch is working.
- 3 When the user selects a country, you will do another fetch to retrieve the photos for that country. This will require adding a `querystring` to the photo URL that contains the `iso` of the country (which can be obtained from the `value` property of the `<select>`). The `querystring` must have the name `iso`; for instance, to retrieve the photos from Canada, the `querystring` would be: `iso=CA`.

The fetched photo data has a `filename` property that can be appended to the supplied image URL and used for the `src` attribute of the generated `<img>`. Set the `title` and `alt` attributes of the image to the photo's `title` property.

Display the second loading animation while the second fetch is working. Be sure to also empty the `<section>` element before adding photos to it.

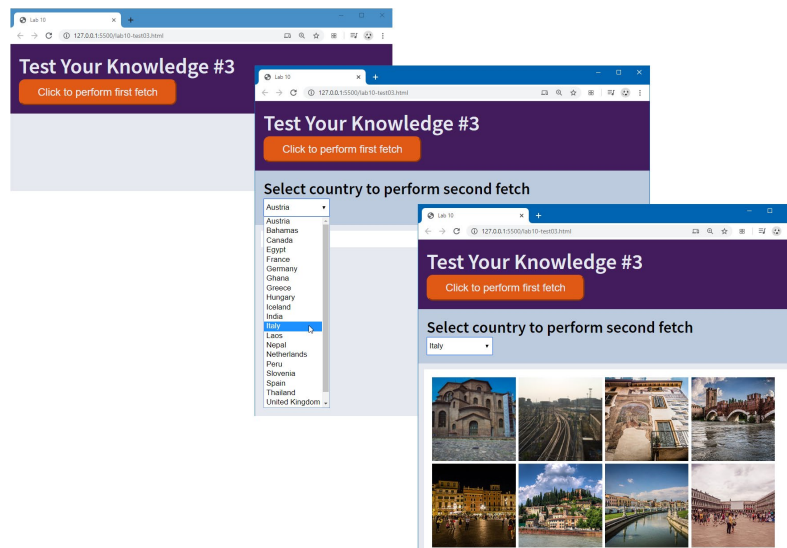


Figure 10.6 – Completed Test Your Knowledge #3

EXERCISE 10.9 — CREATING PROMISES

- 1 Examine `lab10-ex09.html` in the browser. You will be creating promises to simulate working with asynchronous cloud services.
- 2 Modify `lab10-ex09.js` by adding the following:

```
function saveInCloud(filename) {
  // some sample filenames to simulate existing files
  const existingFiles = ['cats.jpg', 'family.jpg',
    'afewdogs.jpg', 'farm.jpg'];

  return new Promise( (resolve, reject) => {
    console.log('saving to cloud ...');
    // to simulate time delay in working with external
    // service, do the next steps after a time delay
    setTimeout( () => {
      // just have a made-up AWS url for now
      let cloudURL = `http://bucket.s3-aws-
region.amazonaws.com/makebelieve/${filename}`;
      // see if passed filename exists
      if ( existingFiles.some( file => file == filename ) ) {
        resolve(cloudURL)
      } else {
        reject( new Error(`${filename} does not exist`));
      }, 2000);
    }
  )
};
```

3 Use this function by adding the following:

```
function simplePromise(filename) {
  saveInCloud(filename)
    .then( url => {
      console.warn(`Cloud URL for ${filename} is ${url}`);
    })
    .catch(err => {
      console.error(err);
    });
}
```

4 Test. Be sure to have the console visible before clicking buttons.

Right now the last two buttons don't do anything. The next steps will chain several promises together.

5 Add the following functions:

```
function tagImageContent(url) {
  return new Promise ( (resolve, reject) => {
    console.log(`Using ML to tag ${url} ...`);
    // list of sample tags
    const tags = ['fun', 'animal', 'cute'];
    // for now simply resolve after a 3 second delay
    setTimeout( () => resolve(tags), 3000);
  });
}

function createThumbnail(url) {
  return new Promise( (resolve, reject) => {
    console.log(`Creating thumbnail of ${url} ...`);
    // simply resolve after a second with a fake CDN url
    const thumbURL =
      "https://res.cloudinary.com/myapp/totallyfake.jpg";
    setTimeout( () => resolve(thumbURL), 1000);
  } );
}
```

6 Use these functions by adding the following:

```
function chainedPromises(filename) {
  saveInCloud(filename)
    .then( url => {
      console.warn(`Cloud URL for ${filename} is ${url}`);
      return tagImageContent(url);
    })
    .then( tags => {
      console.warn('Returned tags=' + tags);
      return createThumbnail('???');
    })
    .then ( url => {
      console.warn('Thumbnail url=' + url);
    })
    .catch(err => {
      console.error(err);
    });
}
```

Why do the first two then() callbacks return another function? Remember that the then() function

should always return a Promise. We want these two actions (create thumbnail and create thumbnail) to happen after the saveInCloud, so they are chained together.

7 Test the chained button.

We still have one problem. The createThumbnail() function needs the cloud URL returned from the resolved saveInCloud. We can solve this simply by saving the value in a variable outside the function.

8 Add the following and test.

```
function chainedPromises(filename) {
  let cloudURL;
  saveInCloud(filename)
    .then( url => {
      console.warn(`Cloud URL for ${filename} is ${url}`);
      cloudURL = url;
      return tagImageContent(url);
    })
    .then( tags => {
      console.warn('Returned tags=' + tags);
      return createThumbnail(cloudURL);
    })
    .then( url => {
      console.warn('Thumbnail url=' + url);
    })
    .catch(err => {
      console.error(err);
    });
}
```

9 The order of tagImageContent and createThumbnail doesn't matter in this example. As such, we could use Promise.all. Edit the code as follows and test using the last button.

```
function simultaneousPromises(filename) {
  saveInCloud(filename)
    .then( url => {
      console.warn(`Cloud URL for ${filename} is ${url}`);
      Promise.all([ tagImageContent(url), createThumbnail(url) ])
        .then(resolves => {
          // do something with array of responses
          const [tags, thumbURL] = resolves;
          console.warn(`Returned tags=${tags}
                        thumbnail=${thumbURL}`)
        })
        .catch(err => {
          console.error(err);
        });
    })
    .catch(err => {
      console.error(err);
    });
}
```

EXERCISE 10.10 — ASYNC AND AWAIT

- 1 Test `lab10-ex10.html` in the browser. You will be refactoring the fetches used in this exercise to use `async ... await` instead.
- 2 In `lab10-ex10.js` comment out the existing `displayCountries()` and create a new one as follows (you can copy and paste most of this code from the existing function).

```
function displayCountries() {
  const countries = getCountryData();

  filters.style.display = "block";
  countries.forEach( c => {
    const opt = document.createElement('option');
    opt.setAttribute('value',c.iso );
    opt.textContent = c.name;
    select1.appendChild(opt);
  });
  select1.addEventListener('input', displayCities);
}
```

- 3 Create the following new function:

```
async function getCountryData() {
  const response = await fetch(countryAPI);
  const data = await response.json();
  return data;
}
```

- 4 Test. It won't work.

Examine the console to see the error message. The problem is that countries is null.

- 5 Add in the following console messages and test.

```
function displayCountries() {
  console.log('before getCountryData');
  const countries = getCountryData();
  console.log('after getCountryData');

  filters.style.display = "block";
  countries.forEach( c => {
    const opt = document.createElement('option');
    opt.setAttribute('value',c.iso );
    opt.textContent = c.name;
    select1.appendChild(opt);
  });
  select1.addEventListener('input', displayCities);
}

async function getCountryData() {
  const response = await fetch(countryAPI);
  console.log('after first await');
  const data = await response.json();
  console.log('after second await');
  return data;
}
```

It is important to remember that the async function is asynchronous; the waiting only

happens *within* the async function itself. That is, the next line after the call to `getCountryData` will get called after the `await` line in it is called, but before it is satisfied.

- 6 Refactor the code as follows by replacing it with the following. Test.

```
async function displayCountries() {
  const countries = await getCountryData();

  filters.style.display = "block";
  countries.forEach( c => {
    const opt = document.createElement('option');
    opt.setAttribute('value',c.iso );
    opt.textContent = c.name;
    select1.appendChild(opt);
  });
  select1.addEventListener('input', displayCities);
}
```

- 7 What about the missing `.catch()`? You can instead use normal JavaScript `try...catch` instead. Add the following and test.

```
async function displayCountries() {
  try {
    const countries = await getCountryData();

    filters.style.display = "block";
    countries.forEach( c => {
      const opt = document.createElement('option');
      opt.setAttribute('value',c.iso );
      opt.textContent = c.name;
      select1.appendChild(opt);
    });
    select1.addEventListener('input', displayCities);
  }
  catch (err) {
    console.error(err);
  }
}
```

- 8 You can also use `async` with anonymous functions. Copy the function code from step 6 and paste it in the modification shown below. It should work as well.

```
button.addEventListener('click', async () => {
  try {
    const countries = await getCountryData();

    filters.style.display = "block";
    countries.forEach( c => {
      const opt = document.createElement('option');
      opt.setAttribute('value',c.iso );
      opt.textContent = c.name;
      select1.appendChild(opt);
    });
    select1.addEventListener('input', displayCities);
  }
  catch (err) {
    console.error(err);
  }
});
```

- 9 Modify `displayCities` to use `async...await`.

TEST YOUR KNOWLEDGE #4

In this exercise, you will extend Test Your Knowledge #3. As you can see in Figure 10.7, this exercise will contain four `<select>` elements, which will be populated from four APIs. You will also use `async...await`.

- 1 Examine `lab10-test04.js` and test out in the browser the URLs of the first five APIs included in the starting code.
- 2 These API fetches are unrelated so they can happen in parallel using `Promise.all()`. Use this method in conjunction with `async...await`. Once the data is retrieved, sort each data set using the `name` or `lastName` (for users) property.
- 3 Populate the four select lists using the following properties from the retrieved data:
 - Continents: `code` (value attribute of each option), `name` (textContent of each option)
 - Countries: `iso` (value attribute of each option), `name` (textContent of each option)
 - Cities: `id` (value attribute of each option), `name` (textContent of each option)
 - Users: `id` (value attribute of each option), `lastName` (textContent of each option)
- 4 Add event handlers for the `input` event of each select. When the user chooses an item from one of the lists, then use the photo API to retrieve photos for the specified continent, country, city, or user. You will supply a different `querystring` parameter based on which filter criteria to use. For instance:

```
images.php?continent=NA
images.php?iso=CA
images.php?city=252920
images.php?user=2
```

- 5 Once you have retrieved the images, display them in the same manner as in Test Your Knowledge #3.

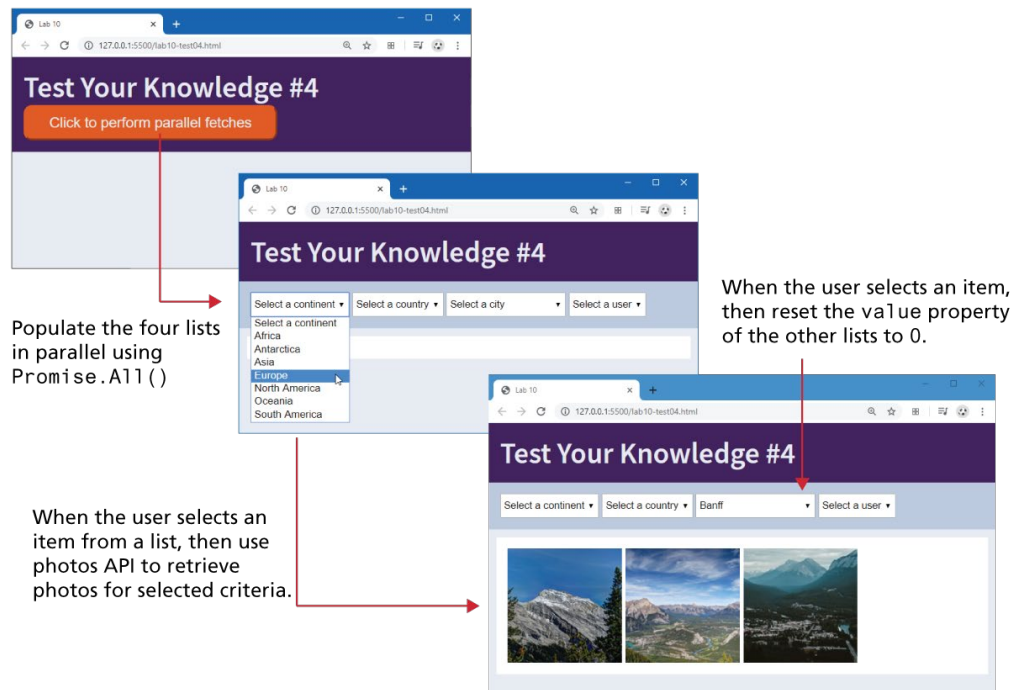


Figure 10.7 – Completed Test Your Knowledge #4

BROWSER APIs

In the last section, you learned how to use the `fetch()` method to access data from external APIs. In this section, you will instead make use of the browser APIs (often also called Web APIs). These are APIs available to JavaScript developers that are provided by the browser.

In recent years, the amount of programmatic control available to the JavaScript developer has grown tremendously. You can now, for instance, retrieve location information, access synthesized voices, recognize and transcribe speech, and persist data content in the browser's own local storage.

In the next exercise you will work with one of these APIs, the Web Storage API, which provides mechanisms for preserving non-essential state across requests and even across sessions. Local Storage is simply a dictionary of strings that lasts until removed from the browser. Session storage is also a dictionary of strings but only lasts as long as the session.

EXERCISE 10.11 — USING THE STORAGE API

- 1 Some of the functionality has already been implemented in the starting files. Examine `lab10-ex11.html` in the browser and then examine in the editor. Try defining a color scheme by clicking on the color selectors and then clicking the Add to Scheme Collection button. This should add a series of appropriate colored boxes in the top part of the page. You can add multiple schemes if you wish (see Figure 10.6)

To allow this exercise to focus on the Storage API, most of the functionality on this page has already been implemented in JavaScript. This JavaScript makes use of several techniques covered earlier in this chapter.

- 2 Try clicking on one of the Test Color Scheme links. It should take you to a page that displays an alert that says “Colors not set yet”. Click Ok and then click on the Return to Scheme Collection link. **Your color schemes should have disappeared.**

This shouldn't be a surprise: the page is using JavaScript to dynamically add content to the page; so if we leave the page, then the page reverts to its initial state. This exercise is about using local storage in the browser to preserve state.

- 3 Edit `js/lab10-ex11.js` by modifying the following code (don't test yet).

```
document.addEventListener("DOMContentLoaded", function () {
  // define a few variables referencing key interface elements
  const colorChoosers =
    document.querySelectorAll('input[type=color]');
  const colorLabels = document.querySelectorAll(
    'fieldset span');
  const schemePreviews =
    document.querySelectorAll('.scheme-group .scheme');
  const schemeGroup = document.querySelector(
    'article.scheme-group');

  // holds collection of user-created schemes ...
  let schemeCollection = retrieveStorage();
```

In the starting code the scheme collection is simply an empty array. In the next step we will modify this method.

- 4 Modify the following code (don't test until instructed later).

```
// retrieve from storage or return empty array if doesn't exist
function retrieveStorage() {
  return JSON.parse(localStorage.getItem('schemes'))
    || [];
}
```

As the comment indicates, this function will either return whatever string is stored in localStorage for the specified key (in this case the key is named 'schemes'). If the key value doesn't exist, then it returns an empty array instead.

The `JSON.parse()` method turns the JSON string (remember that Local Storage can only contain strings) into a JavaScript object.

5 Modify the following code.

```
// update storage with revised collection
function updateStorage() {
    localStorage.setItem('schemes',
        JSON.stringify(schemeCollection));
}
```

This code saves the array of schemes in local storage. Since Local Storage can only save strings, you have to use the `JSON.stringify()` method to convert the array of scheme objects into a JSON string.

6 Modify the following code.

```
// removes collection from storage
function removeStorage() {
    localStorage.removeItem('schemes');
}
```

Once something is added into local storage, it stays until user removes it (by clearing browser history) or by programmatically removing it as shown in this step.

7 Add the following code to the `setupAddSchemeHandler()` method:

```
// add scheme to collection array
schemeCollection.push(scheme);
// update storage with revised collection
updateStorage();
// tell scheme collection to update its display
updateSchemePreviews();
```

When the user clicks the Add to Scheme Collection button, then the method creates a new scheme object populated from the values in the color selectors. It then adds/pushes this new object to the scheme collection array. The new line we added calls your revised update method which adds (or replaces if it already exists) the collection to local storage and then redraws the preview sections.

- 8** Now test by repeating steps 1 and 2. The Test Color Scheme page, which reads the scheme collection from local storage, should show a larger preview of the colors in the scheme. When you return to the main page via the Return to Scheme Collection link, the previously constructed schemes should still be there.
- 9** Close the browser window or tab used for your testing. Restart the browser or create a new tab and navigate to the `lab10-ex11.html` page. **The schemes should still be there.**

- 10** Add the following code to the `setupRemoveAllHandler()` method:

```
// defines handler for Remove All Schemes button
function setupRemoveAllHandler() {
  document.querySelector('button#btnRemoveAll')
    .addEventListener('click', (e) => {
      // empty the scheme collection
      schemeCollection = [];
      // update local storage and update preview display
      removeStorage();
      updateSchemePreviews();
    });
}
```

- 11** Test the Remove All Schemes button. Your schemes should now be gone. Try repeating step 9 (close browser and then re-request).
- 12** Change the `updateStorage()`, `retrieveStorage()`, and `removeStorage()` methods to use `sessionStorage` instead of `localStorage`. You will also have to change the one reference to `localStorage` in `lab10-ex11-tester.js` to `sessionStorage` as well.

With `sessionStorage`, the values only last as long as the browser session.

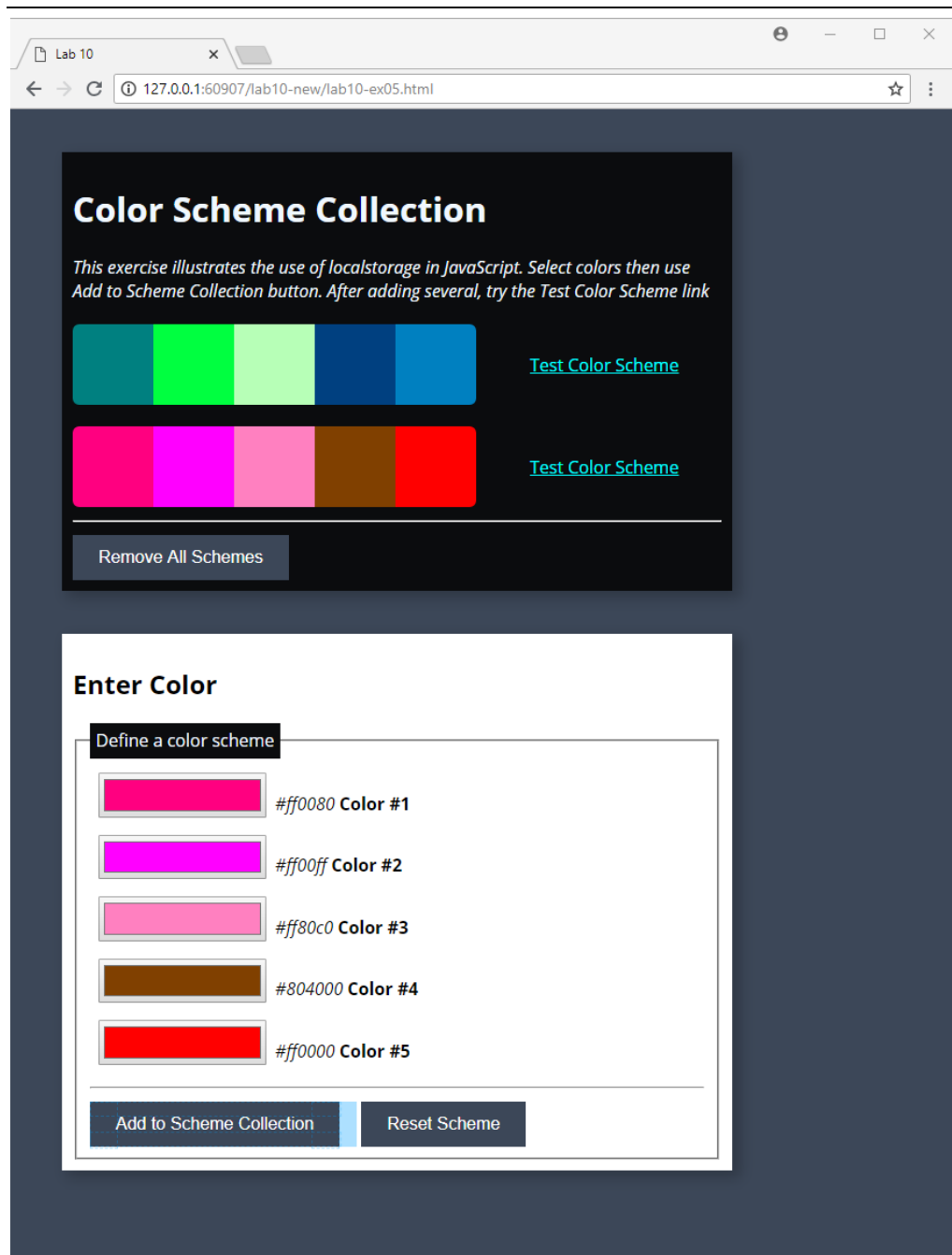


Figure 10.8 – Completed Exercise 10.11

EXERCISE 10.12 — USING SPEECH SYNTHESIS

- 1 Examine `lab10-ex12.html` and then edit `js/lab10-ex12.js` by adding the following code. Save then test by clicking on the Speak button (you will need to have your device's sound turned on).

```
document.addEventListener("DOMContentLoaded", function () {
  document.querySelector('#speak').addEventListener('click',
    () => {
      const utterance =
        new SpeechSynthesisUtterance('Hey there buddy');
      speechSynthesis.speak(utterance);
    });
});
```

The browser should speak the text passed to the `SpeechSynthesisUtterance` constructor.

- 2 Comment out the code you just added.
- 3 Add the following code, which displays the voices available to your browser and operating system. Test in browser and examine the console.

```
window.speechSynthesis.addEventListener('voiceschanged',
  function () {
    let voices = this.getVoices();
    voices.forEach( (voice) => {
      console.log(voice.name + " - " + voice.lang)});
  });
```

- 4 Modify the event handler so it populates the Select A Voice `<select>` list with the available English voices and test.

```
window.speechSynthesis.addEventListener('voiceschanged', () => {
  let select = document.querySelector("#voices");
  // get array of available voices for this computer+browser
  let voices = this.getVoices();
  // filter them so only have the english voices
  let englishVoices = voices.filter( voice =>
    voice.lang.includes('en'));
  // populate the list only the first time in
  if (select.childElementCount == 1) {
    englishVoices.forEach( (voice) => {
      let opt = document.createElement("option");
      opt.setAttribute("value", voice.name)
      opt.textContent = voice.name + ' [' + voice.lang + ']';
      select.appendChild(opt);
    });
  }
});
```

*You should now have a list populated with available voices. Currently, this is listed as experimental **and the voice code listed here only works in Chrome***

- 5 Now add the handler for the Speak button within the `voiceschanged` handler, then test.

```
// now add event handler for speak button
document.querySelector('#speak').
    addEventListener('click', (e) => {

    // get the text to say and the voice options from form
    let message = document.querySelector('textarea').value;
    let selectedVoice = document.querySelector('#voices').value;
    // create utterance and give it text to speak
    let utterance = new SpeechSynthesisUtterance(message);
    // set the speech options (voice, rate, pitch)
    utterance.voice = englishVoices.find(voice =>
        voice.name === selectedVoice);
    utterance.rate = document.querySelector('#rate').value;
    utterance.pitch = document.querySelector('#pitch').value;
    // all ready, make it speak
    window.speechSynthesis.speak(utterance);
});
```

It should work (in Chrome)!

EXTERNAL APIs

Not every external API is simply a service for requesting data. The Google Maps JavaScript API is a prominent example of an API which consists almost entirely of an external `<script>` library; using the API thus involves using its objects' properties and methods.

This section illustrates two different external APIs that do similar things: Google Maps and MapQuest Maps. Many external APIs require you to get some type of API key from the owners of the API. This key is then used with your requests. Both of these examples requires keys

The process of getting a Google Maps JavaScript API key changed in 2018 and 2021. It is possible that when you work on this lab, the process will have changed again. At the time of writing, you will need to first create a Google Cloud Platform account via <https://cloud.google.com/free/> and then create a key. **Be warned: this process can be confusing!** I find that about a quarter of my students struggle with registering and creating the Google Cloud API key. **If you are not having any luck, skip this exercise and try the MapQuest version (Exercise 10.14) instead. If the MapQuest version isn't working (their API requirements also change), skip it also. Welcome to the world of web development, where working code doesn't work 6 months later!**

EXERCISE 10.13 — WORKING WITH GOOGLE MAPS

- 1 You will need to join Google Cloud Platform and then get a Google API key for this project. The process for this can be found at:

<https://developers.google.com/maps/documentation/javascript/get-api-key>

It may take a few hours for you to complete this process. As well, the process of creating a key can be a bit complicated, and seems to change every few years.

- 2 Once you have a Google API key, then edit `lab10-ex13.html` by adding the following:

```
<h1>Populating a Google Map</h1>
<div id="map"></div>
```

Google Map requires an empty `<div>` element. It will eventually be populated with the map graphics.

- 3 Continue editing the html by adding the following to the **end of the body** (it is at the end so as to not require DOM load checking).

```
<script src=
"https://maps.googleapis.com/maps/api/js?key=YOUR_API_HERE&
callback=initMap"></script>
```

Notice that you will have to replace `YOUR_API_HERE` with your actual API, which you should find in your Google Console.

Notice as well the callback parameter in the URL. This is the name of the function to be called when the API is ready to draw your map. You will define this function below.

- 4 Edit `css/lab10-ex13.css` by adding the following:

```
#map {
    height: 600px;
}
```

Perhaps the most common mistake one makes with Google Maps is to forget to set a non-zero height for the element which will contain the map. If you forget this step, then the map will not appear!

- 5 Edit `js/lab10-ex13.js` by adding the following:

```
// Creates map
let map;
function initMap() {
    map = new google.maps.Map(document.getElementById('map'), {
        center: {lat: 41.89474, lng: 12.4839},
        zoom: 6
    });
}
```

The `initMap()` function is quite straightforward. It uses the `Map` constructor and passes it the node that will house the map, the latitude and longitude of the center point of the map, and the zoom setting (a value between 1-20).

6 Test. It should work.

7 Modify the map options as follows (case is important), then test.

```
function initMap() {
  map = new google.maps.Map(document.getElementById('map'), {
    center: {lat: 41.89474, lng: 12.4839},
    mapTypeId: 'satellite',
    zoom: 18
  });
}
```

8 Add the following call to the same method after creating the map:

```
map.setTilt(45);
```

9 Return the `initMap()` function back to how it was in Step 5. Then add the following method:

```
function createMarker(map, latitude, longitude, city) {
  let imageLatLng = {lat: latitude, lng: longitude};
  let marker = new google.maps.Marker({
    position: imageLatLng,
    title: city,
    map: map
  });
}
```

This function will be used to add markers to our map.

10 Now add a call to this function in `initMap()` after creating the map:

```
createMarker( map, 41.89474, 12.4839, "Rome" );
```

11 Comment out this call. You will now get a list of cities from an external API and display a marker on the map for each one. Your starting code has a URL assigned to the variable called `endpoint`. Copy the URL in the string and view in browser to see the structure of the JSON data returned by the API.

12 Add the following code after the endpoint declaration and test.

```
const endpoint = ...
// to simplify no error handling for initial response
fetch(endpoint)
  .then( response => response.json() )
  .then( (data) => {
    // data received, now add cities to map
    data.forEach( (city) => {
      createMarker( map, city.Latitude,
        city.Longitude, city.AsciiName );
    });
  })
  .catch( error => console.log(error) );
```

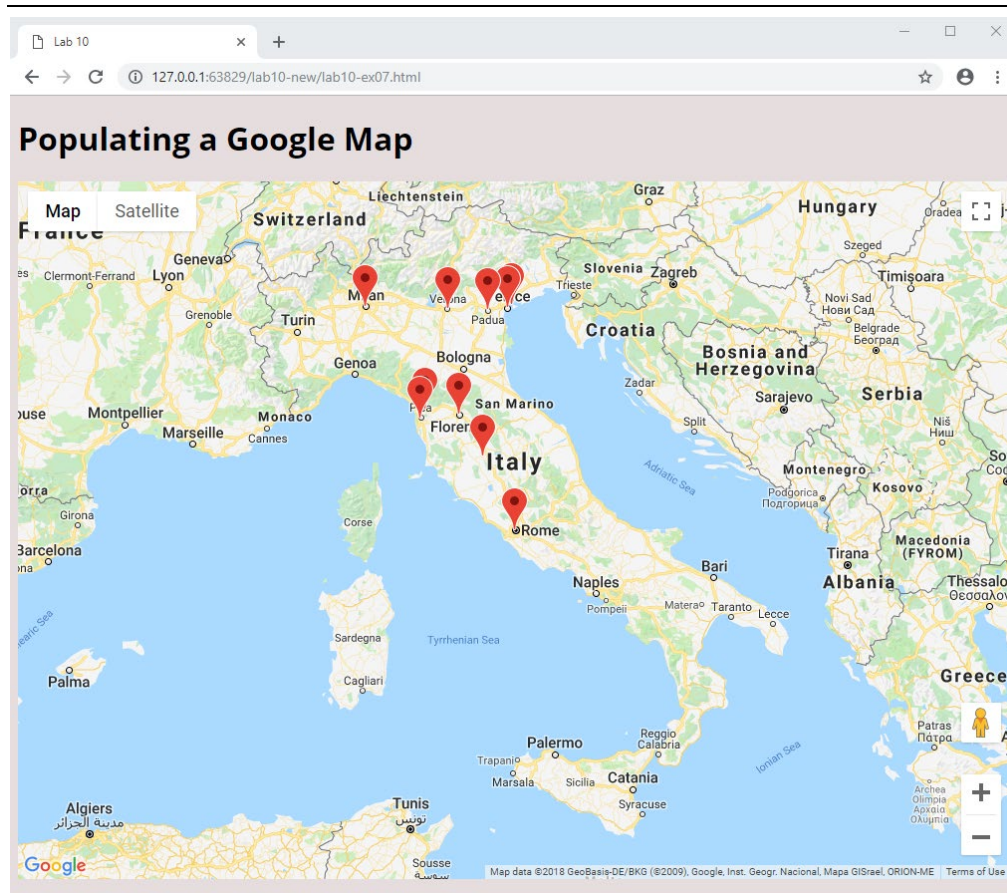


Figure 10.9 – Exercise 10.13

Note: Be careful with your Google Map API key! If others gained access to it and used it for their own site, you potentially would be financially responsible for their usage.

Getting an API key with MapQuest developer is pretty straightforward. First, create a developer account via <https://developer.mapquest.com>. Choose the Free account! You will then create a new key via Manage Keys option, and make use of the consumer key value.

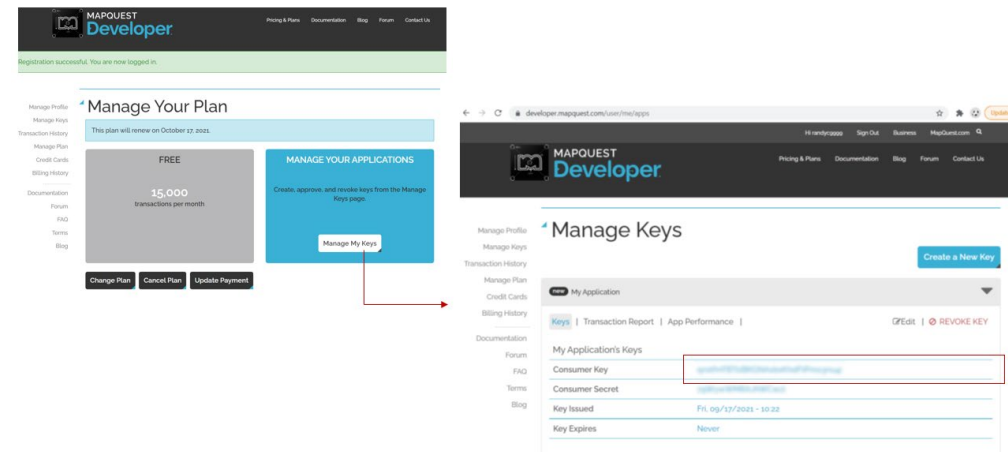


Figure 10.10 – Creating an API key for MapQuest

MapQuest makes use of the Leaflet.js mapping library. To learn more about how to manipulate these maps, visit <https://leafletjs.com/reference.html>.

EXERCISE 10.14 — WORKING WITH MAPQUEST

- 1 You will need to join the MapQuest developer program via <https://developer.mapquest.com>. When you login, you can create a new API key (or view any existing ones).

Note: this exercise is functionally identical to the previous one. It uses MapQuest instead of Google Maps.

- 2 Once you have a MapQuest API key, then edit `lab10-ex14.html` by adding the following:

```
<script src=
  "https://api.mqcdn.com/sdk/mapquest-js/v1.3.2/mapquest.js">
</script>
<link type="text/css" rel="stylesheet" href=
  "https://api.mqcdn.com/sdk/mapquest-js/v1.3.2/mapquest.css"/>
```

Check the Getting Started documentation

(<https://developer.mapquest.com/documentation/mapquest-js>). These URLs may have been updated (and should be there to copy-and-paste).

3 Add the following:

```
<h1>Populating a MapQuest Map</h1>
<div id="map"></div>
```

MapQuest requires an empty <div> element. It will eventually be populated with the map graphics.

4 Edit `css/lab10-ex14.css` by adding the following:

```
#map {
  height: 600px;
}
```

Perhaps the most common mistake one makes with MapQuest is to forget to set a non-zero height for the element which will contain the map. If you forget this step, then the map will not appear!

5 Edit `js/lab10-ex14.js` by adding the following:

```
document.addEventListener("DOMContentLoaded", function() {
  L.mapquest.key = 'replace with your API key';

  const map = L.mapquest.map('map', {
    center: [41.89474, 12.4839],
    layers: L.mapquest.tileLayer('map'),
    zoom: 12
  });

  map.addControl(L.mapquest.control());
});
```

6 Test. It should display a map of Rome. Change the `zoom` property to 6 and test. It should display all of Italy.**7** Add the following method:

```
function createMarker(map, latitude, longitude, city) {
  L.marker([latitude, longitude], {
    icon: L.mapquest.icons.marker(),
    draggable: false
  }).bindPopup(city).addTo(map);
}
```

This function will be used to add markers to our map.

8 Add the following function call before the call to `map.addControl`.

```
createMarker( map, 41.89474, 12.4839, "Rome" );
```

9 Comment out this call. You will now get a list of cities from an external API and display a marker on the map for each one. Your starting code has a URL assigned to the variable called `endpoint`. Copy the URL in the string and view in browser to see the structure of the JSON data returned by the API.

- 10 Add the following code after the endpoint declaration and test.

```
const endpoint = ...
// to simplify no error handling for initial response
fetch(endpoint)
  .then( response => response.json() )
  .then( (data) => {
    // data received, now add cities to map
    data.forEach( (city) => {
      createMarker( map, city.Latitude,
                    city.Longitude, city.AsiiName );
    });
  })
  .catch( error => console.log(error) );
```

- 11 Finally, to add some interactivity, the following code will pan to and zoom into the city when you click on a marker.

```
function createMarker(map, latitude, longitude, city) {
  L.marker([latitude, longitude], {
    icon: L.mapquest.icons.marker(),
    draggable: false
  }).bindPopup(city).addTo(map).on('click', function() {
    markerLocation = this.getLatLng();
    map.flyTo([markerLocation.lat, markerLocation.lng],12);
  });
}
```

Note: Be careful with your MapQuest API key! If others gained access to it and used it for their own site, you potentially would be financially responsible for their usage.

EXERCISE 10.15 — CHARTING WITH GOOGLE CHARTS

- 1 Google Charts is a free external API that doesn't require an API key or registering. You can simply use it. You can find the documentation for Google Charts at <https://developers.google.com/chart/interactive/docs>.

- 2 Examine `lab10-ex15.html` and `lab10-ex15.js`.

In this exercise you will be creating three charts. The data arrays for the charts are already defined within the provided JavaScript file.

- 3 Add the following to the top of the `<head>` element in `lab10-ex15.html`.

```
<head>
  <script type="text/javascript"
    src="https://www.gstatic.com/charts/loader.js"></script>
  <link rel="stylesheet" href="css/lab10-ex15.css">
  <script src="js/lab10-ex15.js"></script>
</head>
```

It's possible that Google might change the URL of its charting API. You might need to check the documentation to ensure that this is still the current URL.

- 4 Add the following function to `lab10-ex15.js`.

```
function drawColumnChart() {
  const data =
    google.visualization.arrayToDataTable(salesArray);

  const columnDiv = document.querySelector('#columnChart');
  const chart = new google.visualization.ColumnChart(columnDiv);
  chart.draw(data);
}
```

- 5 Add the following code and test.

```
document.addEventListener('DOMContentLoaded', () => {

  google.charts.load('current', {'packages':['corechart']});
  google.charts.setOnLoadCallback(drawColumnChart);
});
```

- 6 Add the following function to `lab10-ex15.js`.

```
function drawPieChart() {
  const data =
    google.visualization.arrayToDataTable(marketArray);

  const pieDiv = document.querySelector('#pieChart');
  const chart = new google.visualization.PieChart(pieDiv);
  chart.draw(data);
}
```

- 7 Add the following function to `lab10-ex15.js`.

```
function drawScatterChart() {
  const data =
    google.visualization.arrayToDataTable(revenueArray);

  const div = document.querySelector('#scatterChart');
  const chart = new google.visualization.ScatterChart(div);
  chart.draw(data, options);
}
```

- 8 Add the following function to `lab10-ex15.js`.

```
function drawCharts() {
  drawColumnChart();
  drawPieChart();
  drawScatterChart();
}
```

- 9 Modify the following line and test.

```
google.charts.setOnLoadCallback(drawCharts);
```

You should now see three charts.

- 10 Modify `drawColumnChart()` as follows.

```
function drawColumnChart() {
  const data = google.visualization.arrayToDataTable(salesArray);

  const options = {
    hAxis: {
      title: 'Month',
      titleTextStyle: {color: '#333'},
      textStyle: {color: '#666'}
    },
    vAxis: {
      title: 'Amount (thousands $)',
      titleTextStyle: {color: '#333'},
      textStyle: {color: '#666'},
      format: '$#,###'
    },
    colors: ['#4285f4', '#ea4335'],
    backgroundColor: 'transparent',
    legend: {
      position: 'right',
      maxLines: 3,
      textStyle: {color: '#333', fontSize: 12}
    },
    chartArea: {
      left: 60,
      top: 10,
      width: '80%',
      height: '70%'
    }
  };
};
```

```
const columnDiv = document.querySelector('#columnChart');  
const chart = new google.visualization.ColumnChart(columnDiv);  
chart.draw(data,options);  
}
```

11 Test.

Much of the documentation is devoted to describing all the different ways to customize the chart via the `options` element.

12 Add the following property to the `options` object and test.

```
animation: {  
  startup: true,  
  duration: 1000,  
  easing: 'out'  
}
```

13 Experiment with the options and if interested, add options to the other charts.

TEST YOUR KNOWLEDGE #5

In this exercise, you will use `fetch` to retrieve a JSON file containing multiple datasets. The user will be able to select a dataset and then display its data in a pie chart.

- 1 Examine the provided JSON data file (`pieData.json`). Use the technique from Exercise 10.4 to fetch the JSON file. Do steps 2-5 after the fetched data has been loaded.
- 2 Populate the `<select>` element using the `title` property as the label and the `key` property as the value for each `<option>` element.
- 3 Add an event handler for the `change` event of the `<select>` element that will draw the chart (see Exercise 10.15). In the starting file, you have been provided with an `options` element within a comment that you can use to achieve a chart styled similar to that shown in Figure 10.11.
- 4 When a new data set is selected, also update the `data-info` `<div>` element so that it displays the `title` and `description` properties from the selected dataset.
- 5 Set the `colors` property of the chart `options` element to the `colors` property of the selected dataset.

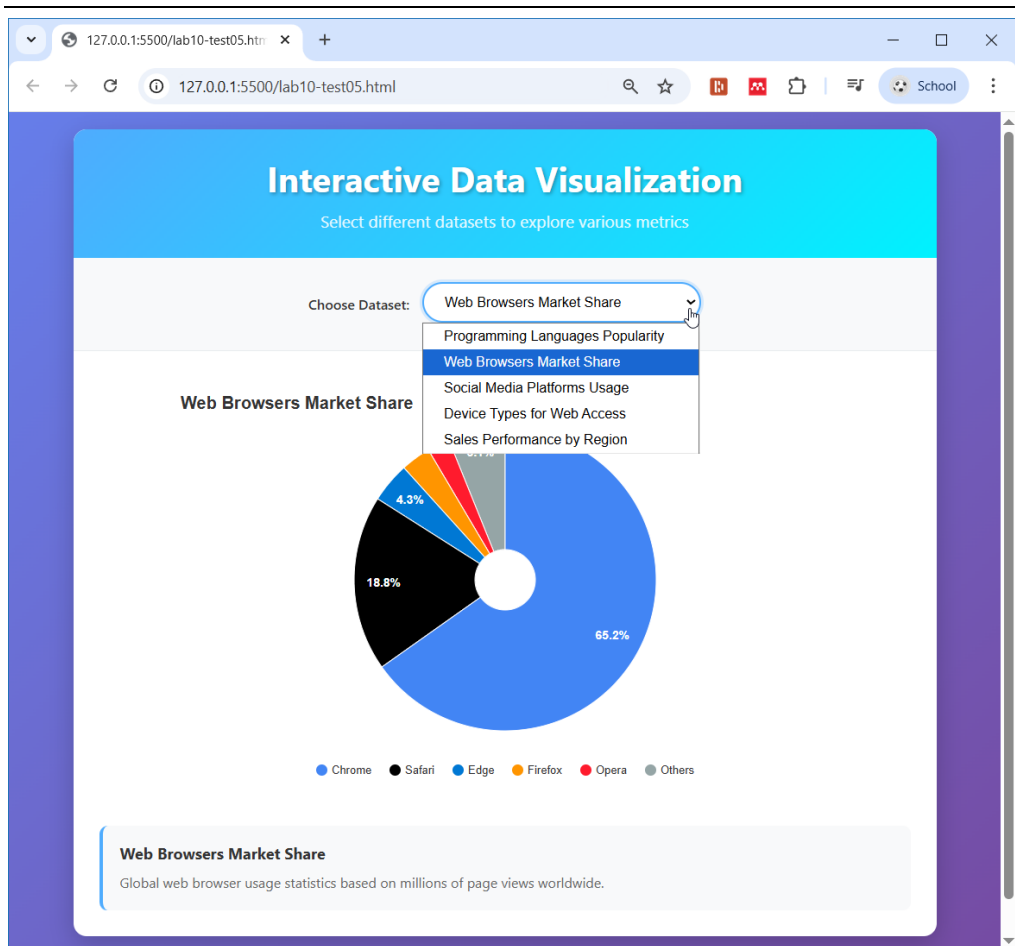


Figure 10.11 – Finished Test Your Knowledge #5